EPMDynamoCode tutorial

version 0.1

Philippe Marti

March 26, 2013

Contents

1 External requirements

- Git
- GCC
- FFTW3
- LAPACK
- HDF5 (with parallel IO for MPI code)
- CMake

2 Getting the sources

2.1 Initial download

The easiest way to get the latest version of the code the first time is to clone the git repository:

```
# $>git clone gitosis@pmarti.nine.ch:EPMDynamoCode.git
```

This repository is not publicly available and the above command will only work if your SSH key has been authorized (contact philippe.marti@colorado.edu). You should now have the *EPMDynamoCode* directory. Hence forth, this main source directory will be refered to as the git directory. There are some external dependencies that are included in the code tree. To simplify migration to other versions, these are included as submodules. To get these dependencies installed, the submodules have to be initialised and updated:

```
# $>cd EPMDynamoCode
# $>git submodule init
# $>git submodule update
```

All sources are now be readily avaiable on your machine.

2.2 Updates

If you already downloaded an earlier version, the git old git directory can be updated to the latest version of the code by doing a "pull" from the old git directory:

```
# $>cd EPMDynamoCode
# $>git pull
```

3 Directory structure

3.1 include and src

The *include* and *src* directories contain the main part of the source code.

3.2 Programs

The *Programs* directory contains the actual executables. The most important ones are *SimulationRun*, *GenerateState*, *GenerateSource* and *GenerateCSCS*.

- *SimulationRun* is a "script"-like binary that is used by all the different simulation implementations, more details will be given later.
- *GenerateState* is used to generate initial states for the different simulations.
- *GenerateSource* is used to generate the internal heat source state.
- *GenerateCSCS* converts the spectral state files into an HDF5 format that can be read into ParaView and Visit on the CSCS clusters.

3.3 Clusters

The *Clusters* directory contains a few basic batch scripts for different clusters.

3.4 CodeDoc and Doc

The *CodeDoc* directory is the placeholder for the `doxygen` generated "technical" code documentation. To generate it (or update it if the code changed), a simplified Makefile is provided. Use the following command to generate the HTML and LaTeXdocumentation in the git directory:

```
# >$make CodeDoc
```

The generated documentation is in *CodeDoc*. The *Doc* directory contains some additional documentation also compiled into the technical documentation in *CodeDoc*.

3.5 Config

The *Config* directory contains a few example of configuration files in XML format to use for the different simulations.

3.6 External

The *External* directory contains the additional third party dependencies: Eigen, rapidxml and interfaces to LAPACK routines.

3.7 TestSuite

The *TestSuite* directory contains some small test executables used to test new and old code parts during the development stage.

4 Compilation

The compilation and setup is based on an out-of source build and implemented through `CMake` configuration files.

4.1 Setup

To setup a build directory for a specific architecture, start by creating a build diretory somewhere on your computer:

```
# $>mkdir /path/to/build/MyBuild
```

From within the newly created *MyBuild* directory, start the `CMake` configuration tool `ccmake` by giving it the path to the git directory as argument:

```
# $>cd /path/to/build/MyBuild
# $>ccmake /path/to/sources/EPMDynamoCode
```

If everything went well, your shell should now have been replaced by the `ccmake` configuration tool screen. Type "c" to start the configuration. Errors might appear at that stage, simply ignore them by pressing "e". `CMake` uses an iterative approach to generate the Makefiles, these errors appear because the sources have not yet been configured. The screen should know show two *CMAKE_* options and eight *EPMDYNAMO_* options. Selecting and entry with the keyboard navigation arrows and pressing "enter" allows to change the settings. When a setting selected, accepted values are shown at the bottom of the terminal.

First the two general settings:

- *CMAKE_BUILD_TYPE*: Release (or Debug for debugging)
- *CMAKE_INSTALL_PREFIX*: (Leave it as it is, it's not used)

Now the more interesting, code related settings (the order might be different on your computer):

- *EPMDYNAMO_COMPILER*: GCC (other options might be available but haven't been tested in a while)
- *EPMDYNAMO_MEMORY_USAGE*: (leave blank, or BigMem)
- *EPMDYNAMO_PLATFORM*: Rosa (select the platform you are on: Brutus, Rosa, Core2. Core2 should work for a standard workstation)
- *EPMDYNAMO_RADIAL_GROUPEDCOMM*: ON/OFF (Group data during the "radial" communication, is faster but needs more memory)
- *EPMDYNAMO_SH_GROUPEDCOMM*: ON/OFF (Group data during the "Spherical Harmonic" communication, is faster but needs more memory)
- *EPMDYNAMO_SPLIT_RADIAL*: ON/OFF (Split data among CPUs during radial transform)
- *EPMDYNAMO_SPLIT_SH*: OFF/ON (Split data among CPUs during SH transform)
- *EPMDYNAMO_TESTING*: OFF (leave it to OFF unless you need the TestSuite)

The serial version of the code is obtained by switching off both *EPMDYNAMO_SPLIT_* options. The tubular splitting is obtained by activating both splittings.

There are also a couple of hidden advanced settings that can be changed by pressing "t". The *EPMDYNAMO_FFTW_PLAN_FLAG* allows to change the kind of plan FFTW generates (see FFTW3 documentation). *EPMDYNAMO_SHAREDPTR_IMPLEMENTATION* allows to choose the implementation used for the smart shared pointers. *TR1* (default) should always work nowadays and *Boost* was the one used originally. With recent compilers and *Boost* versions there shouldn't be any difference between the two choices.

Once the configuration is done, hit "c" to run the configuration tool. If everything went well, you should now have the option to press *g* to generate the Makefiles. Due to the iterative approach used by `CMake`, you might need to pressing "c" several times for the *g* option to appear. Finally, generate the Makefiles by pressing "g".

4.2 Compiling a simulation

There are several different simulations that can be compiled. They differ in the equations that are solved and also in the type and number of input files they'll need. The implemented simulations can be seen by listing the content of the *Implementations* subdirectory in the git directory:

```
# $>ls include/Simulations/Implementations/
```

If for example, you want the full dynamo simulation, you would compile it by typing the following in the build directory:

```
# $>make DynamoSimulation
```

or to speedup the compilation if several cores are avaiable use the *-j NCORES* flag for make. For example for 4 parallel compilation jobs

```
# $>make -j 4 DynamoSimulation
```

After successful compilation, the binary will be available under Programs in the build directory.

4.3 Cluster setup: ETHZ Brutus

Before configuring the source and compiling the simulation a few modules need to be loaded on Brutus:

```
# $>module load cmake
# $>module load gcc/4.6.2
# $>module load open_mpi
# $>module load hdf5_para
# $>module load fftw
```

A basic queue submission script *brutus_batch.lsf* for Brutus's LSF can be found in the *Clusters* directory in the git directory. Simply copy the

example file to the directory in which the simulation will run on the cluster, edit it and submit to the queuing system.

Important: Parallel HDF5 needs a parallel file system to work properly. *Do not* run the simulation in your home directory, always run it on the lustre scratch. If you don't IO will be **very slow** and Brutus administrators won't be happy.

Proceed with step 4.2 "Compiling a simulation".

4.4 Cluster setup: CSCS Rosa

Before configuring the source and compiling the simulation a few modules need to be loaded on Rosa:

```
# $>module load PrgEnv-gnu
# $>module load cmake
# $>module load fftw
# $>module load hdf5-parallel
```

A basic queue submission script *rosa_batch.slurm* for Rosa's SLURM can be found in the *Clusters* directory in the git directory. Simply copy the example file to the directory in which the simulation will run on the cluster, edit it and submit to the queuing system.

Important: Parallel HDF5 needs a parallel file system to work properly. *Do not* run the simulation in your home directory, always run it on the scratch. If you don't IO will be **very slow** and CSCS administrators won't be happy.

Proceed with step 4.2 "Compiling a simulation".

5 Running

To run a simulation you will basically need the corresponding binary and a configuration file. The configuration file is an XML file which has to be called *parameters.cfg*. Some examples of configuration files can be found in the *Config* directory in the source code. In addition to this, depending on the actual simulation additional configuration files and state files in HDF5 format might be required.

5.1 Simulation parameters: *parameters.cfg*

The first part of the file between the *general* tags allow the simulation to check that the required files are used. The *type* flag specifies the non-

dimensionalisation type. This normally doesn't require any modification.

The second part, between the *truncation* tags, specifies the truncation and the number of CPUs to use for the simulation. The *symmetry* flags specify a possibly assumed symmetry in the longitudinal direction (**not tested**)

The third part, between the *physical* tags specifies the value of the physical parameters. This section depends on the actual simulation and choice of non-dimensionalisation.

The fourth part, between the *boundary* tags specify the boundary conditions for the different fields. For the codensity, "0" specifies a fixed temperature setup and "1" a fixed flux at the outer boundary. For the velocity field, "0" specifies a non-slip boundary condition and "1" the stress-free boundary condition at the outer boundary. Finally, for the magnetic field "0" specifies an insulating outer boundary. Additional boundary conditions might be available for some simulations.

The fifth part, between the *timestepping* tags specify timestepping related settings. The *time* flag allows to specify an initial time. "-1" makes it start from the stored time in the initial state. An initial timestep can be specified with a negative value, e.g. setting "-1e-5" used an initial timestep of "1e-5". A fixed timestep can be imposed with a positive value for the *timestep* flag. Setting it to "-1" uses the default adaptive timestep scheme.

The last part, between the *run* tags allows to specify run length and IO settings. *maxtime* allows to specify a maximum integration time or infinite time with "-1". *maxtstep* allows to set a maximum number of timesteps, or infinite number of steps with "-1". *arate* and *srate* specify the ASCII and HDF5 state file writing rate respectively, in number of timesteps. *walltime* specifies the maximum wall time of the simulation in hours, for example setting it to 1.5 will stop the simulation after 1h30. In case of large resolutions and/or large number of CPUs this number should be slightly smaller (by several minutes) than the time requested on the cluster. The wall time only takes into account the actual computation time, the finalization steps involving a possibly expensive output writing is not included.

5.2 Examples: DynamoSimulation

To run a dynamo simulation, four files are required: *parameters.cfg*, *state_initial.hdf5*, *source_cod.hdf5* and the binary *DynamoSimulation*. The non-dimensionalisation parameters set in the source code for the *DynamoSimulation* have to match the one used in the configuration file. Once all four files are in the same directory, the simulation can be started by simply executing the binary.

state_initial.hdf5 is an HDF5 file containing the initial state for the sim-

ulation. All used fields have to be present (additional fields in these files will simply be ignored). *source_cod.hdf5* contains the spectral expansion of the internal heat source term.

5.3 Examples: PrecessionDynamoSimulation

To run a precession dynamo simulation, four files are required: *parameters.cfg*, *optional.cfg*, *state_initial.hdf5* and the binary *PrecessionDynamoSimulation*. The non-dimensionalisation parameters set in the source code for the *DynamoSimulation* have to match the ones used in the configuration file. Once all four files are in the same directory, the simulation can be started by simply executing the binary.

state_initial.hdf5 is an HDF5 file containing the initial state for the simulation. *parameters.cfg* contains the simulation configuration and *optional.cfg* contains additional variables required to setup the precession problem. All used fields have to be present (additional fields in the file will simply be ignored).

6 Post-processing

6.1 ASCII files

Each simulation outputs human readable ASCII files during the simulation. These are useful for a fast diagnostic and deciding if the simulation should keep going or should be aborted. They do all generate output files for the kinetic energy and spectrum *vel_energy.dat* and *vel_spectrum.dat* and if applicable will also produce output files for the magnetic energy and spectrum *mag_energy.dat* and *mag_spectrum.dat*.

The evolution of the timestep length during the simulation is stored in *timestep.dat*.

6.2 State files

Each simulation will periodically write full simulation state files called *state####.hdf5* with #### an increasing number. These files store the complete spectral expansion of all relevant scalar and vector fields. Copying any of these files to *state_initial.hdf5* will allow to restart the simulation from this state.

6.3 Visualization

The tool *GenerateCSCS* can be used to visualize the values of the scalar and vector fields in physical space. Which fields should be written to the output HDF5 file can be set in *GenerateCSCS* allowing for example to only output the toroidal component of the velocity field. *GenerateCSCS* expects a HDF5 state file called *state4CSCS.hdf5* and a *parameters.cfg* file (to set the resolution to use) in the directory it is called from. *state4CSCS.hdf5* doesn't have to be a real file, it might also be a link to an actual state file. The output is an HDF5 file called *CSCSParaview0000.hdf5* that can be read directly into `ParaView` and `visit` on the CSCS visualization cluster. To use ParaView locally it has to be installed from source. An additional plugin is also necessary.

6.4 Simulation setup

6.5 SimulationConfig

The configuration file *include/Config/SimulationConfig.hpp* in the git directory contains some additional configuration options. Whenever these are changed the whole simulation has to be compiled anew. The only part that might require any modification is the choice of nondimensionalisation parameters. At the end of the file, in the list of possible nondimensionalisations, only one can be uncommented. Most of the testing was done with *EQRaRoParameters* and *EPmParameters*.

6.6 DynamoSimulation

The nondimensional parameters setup in *SimulationConfig.hpp* in the git directory should be set to *EQRaRoParameters* as this is the one that had most testing.

In most cases no modifications to the top level *include/Simulations/Implementations/DynamoSimulation.hpp* and *src/Simulations/Implementations/DynamoSimulation.cpp* implementation files should be required. Nevertheless, there are two methods that are useful to know. The *initEquations()* method is responsible for the initialisation of the system of equations to solve. The boundary conditions and the corresponding integer flag to put in the XML configuration file are defined there. Adding a new boundary condition or checking what flag is used for a specific boundary condition can be done by checking the definition in *src/Simulations/Implementations/DynamoSimulation.cpp*.

If additional ASCII output is required, additional output files can be added to the *addASCIIOutput()* method.

6.7 PrecessionDynamoSimulation

The nondimensional parameters setup in *SimulationConfig.hpp* should be set to *EPmParameters* as this is the one that had most testing.

In most cases no modifications to the top level *include/Simulations/Implementations/PrecessionDynamoSimulation.hpp* and *src/Simulations/Implementations/PrecessionDynamoSimulation.cpp* implementation files in the git directory should be required. Nevertheless, there are two methods that are useful to now. The *initEquations()* method is responsible for the initialisation of the system of equations to solve. The boundary conditions and the corresponding integer flag to put in the XML configuration file are defined there. Adding a new boundary condition or checking what flag is used for a specific boundary conditions can be done by checking the definition in *src/Simulations/Implementations/PrecessionDynamoSimulation.cpp*.

If additional ASCII output is required, additional output files can be added to the *addASCIIOutput()* method.